

Procédure de déploiement — Passerelle (onboardcc)

Document créé le 07/09/2026. Structuré et complété au fil de la session de déploiement sur o2switch (passerelle.dm-informatique.fr).

 **Valeurs masquées dans cette version publiée** (dépôt GitHub) : `<COMPTE>` = identifiant du compte d'hébergement o2switch, `<HOTE_O2SWITCH>` = nom d'hôte SSH du serveur, `<IP_SERVEUR>` = son adresse IP, `<SERVEUR>` = son nom interne. À remplacer par les vraies valeurs (connues du responsable de l'hébergement) avant de copier une commande.

Objectif de ce document

 **Scindé en 2 documents le 15/09/2026** (le document original mélangeait installation et gestes de maintenance courante, source de confusion pour un lecteur non-initié). **Ce document-ci ne couvre que l'installation initiale ou une réinstallation complète depuis zéro** (nouveau serveur, migration, ou base perdue). Pour tout geste de maintenance courante sur une installation déjà fonctionnelle (déployer un correctif, sauvegarder, diagnostiquer un problème signalé par un utilisateur, changer un mot de passe...), voir [2-procedure_maintenance_passerelle.md](#).

Ce document décrit, étape par étape, la procédure complète pour déployer l'application **Passerelle** (nom technique [onboardcc](#)) sur un hébergement de production, **en partant de rien**.

Il est rédigé à deux niveaux de lecture :

- **Aujourd'hui** : c'est la feuille de route suivie par Damien MOREAU pour déployer sur son hébergement personnel o2switch (sous-domaine passerelle.dm-informatique.fr), dans le cadre du bootcamp Fil Rouge et de la recette utilisateur de la première version de Passerelle.
- **Demain** : c'est la procédure que la DCC (ou tout prestataire technique qu'elle mandatera) pourra suivre pour déployer ou migrer l'application sur son propre serveur — d'où le soin apporté à distinguer ce qui est **spécifique à o2switch** de ce qui est **générique à tout hébergement Node.js + PostgreSQL**.

Chaque étape est documentée avec : ce qu'il faut faire, pourquoi, et comment vérifier que l'étape est bien terminée avant de passer à la suivante.

 **Comprendre** : chaque encadré comme celui-ci apporte une explication de fond sur un concept technique utilisé dans le paragraphe qui précède — pour que ce document reste compréhensible même sans avoir suivi toute la session où il a été écrit.

 **RÈGLE À RESPECTER POUR TOUT MOT DE PASSE DE CE PROJET** (base de données, et par précaution tout autre mot de passe créé pour ce projet) : **uniquement des lettres, des chiffres, et éventuellement _ - .** — jamais d'autres caractères spéciaux (`# ? & @ : / % ! $ ' etc.`).

Pourquoi une règle aussi stricte : un mot de passe généré avec des caractères "forts" classiques a provoqué **deux bugs distincts** le 09/09/2026 — **!** interprété par le terminal (bash) comme un rappel d'historique de commande, puis `#/?/&` cassant la lecture de `DATABASE_URL` comme une URL (`#`=début de fragment, `?`=début de paramètres, `&`=séparateur de paramètres — tous ont un sens réservé dans une URL). Résultat : connexion impossible, message d'erreur générique **Invalid URL** très difficile à

relier à la vraie cause sans un diagnostic poussé (voir §7 et la FAQ "Changer le mot de passe de la base de données" ci-dessous).

Les seuls caractères non alphanumériques **sûrs partout** (terminal, URL, SQL) sont `_ - . —` — s'en tenir à ceux-là évite ce risque entièrement, sans sacrifier la sécurité (la longueur du mot de passe compte bien plus que la diversité des symboles utilisés).

Prérequis

- ☒ Accès à l'espace d'hébergement o2switch — **SSH confirmé disponible** (07/09/2026, cPanel → Sécurité → Accès SSH)
- ☐ Sous-domaine `passerelle.dm-informatique.fr` créé (fait le 06/09/2026)
- ☐ Confirmation que "Setup Node.js App" est disponible en cPanel (confirmé le 06/09/2026)
- ☐ Confirmation que PostgreSQL est disponible en cPanel (confirmé le 07/09/2026)
- ☐ Accès en lecture au dépôt GitHub `onboardcc` (public) — vérifier qu'il est à jour (voir §1.1)

Sommaire des étapes

1. Récupération et vérification du code source
2. Préparation de l'hébergement (Node.js Selector)
3. Base de données PostgreSQL
 - 3.5 Cas particulier : poste de développement local
4. Variables d'environnement de production
5. Stockage des pièces jointes (dossier `uploads/`)
6. Configuration du serveur SMTP (envoi d'emails)
7. Déploiement du code et démarrage de l'application
8. Test de bout en bout en production
 - 8.1 Création du premier compte admin de l'application
 - 8.2 Création des comptes entreprises de test (avec le compte admin)
 - 8.3 Liste des sanity checks à valider avant mise à disposition
9. Point de vigilance pour une future migration DCC

Pour la suite (maintenance courante une fois l'application en service), voir [2-procedure_maintenance_passerelle.md](#).

1. Récupération et vérification du code source

1.1 Vérifier que GitHub est à jour (avant toute récupération)

Le code s'installe **uniquement depuis le dépôt GitHub**

<https://github.com/DamienMoreauConsultant/onboardcc> (branche `main`). Le serveur ne fait que « récupérer » (`git clone` la première fois, puis `git pull`) : il n'écrit jamais de code.

⚠ Rappel avant chaque `git clone` ou `git pull` sur le serveur : vérifier que le dépôt GitHub est bien à jour. Ouvrir la page du dépôt sur github.com et contrôler que le dernier commit (message et date) correspond à la version que l'on veut installer. Un serveur ne peut récupérer que ce qui a été poussé sur GitHub : si le développeur a des modifications non poussées, les lui faire pousser d'abord.

💡 **Comprendre : GitHub est le « maître », tout le reste n'est que lecture.** **push** = envoyer ses commits (sauvegardes) vers GitHub. **pull** = récupérer les derniers commits depuis GitHub sur une machine. Le développeur pousse ; les serveurs (o2switch, futur serveur DCC) ne font que « pull », jamais l'inverse. Cette règle à sens unique garantit qu'aucune synchronisation ne peut rien écraser par erreur.

1.2 Configuration de l'accès SSH au serveur o2switch

Confirmé le 07/09/2026 : l'offre o2switch inclut l'accès SSH, géré depuis cPanel → section **Sécurité** → **Accès SSH** → **Gérer les clés SSH**.

💡 **Comprendre : SSH, un protocole pour piloter un serveur à distance.** SSH (*Secure Shell*) permet d'exécuter de vraies commandes sur le serveur distant (configurer Node.js, interroger la base PostgreSQL, consulter les logs) depuis un terminal sur son PC, à travers une connexion chiffrée — contrairement au FTP, qui ne fait que transférer des fichiers.

💡 **Comprendre : authentification par clé plutôt que par mot de passe.** Une **paire de clés** est générée : la **clé privée** reste uniquement sur le PC qui se connecte (jamais partagée, y compris avec Claude), la **clé publique** est déposée sur le serveur. À la connexion, le serveur vérifie que la clé privée présentée correspond mathématiquement à la clé publique enregistrée — sans qu'aucun mot de passe ne circule. Analogie : la clé privée est la clé physique de la maison (toujours gardée sur soi), la clé publique est la serrure installée côté serveur.

Procédure suivie (07/09/2026) :

1. cPanel → Accès SSH → Gérer les clés SSH → **Générer une nouvelle clé**.
2. Paramètres : nom **passerelle_pc_damien**, type RSA, taille 4096 bits, phrase secrète obligatoire (o2switch impose un niveau de sécurité $\geq 50/100$) — générée via le bouton "Générateur de mots de passe" intégré, conservée uniquement par Damien (jamais partagée).
3. ⚠️ Point d'attention : cPanel génère la paire de clés **côté serveur** (`/home/<COMPTE>/.ssh/`). Le journal affiché après génération (fingerprint, randomart) n'est **pas** la clé elle-même — il faut retourner sur la page "Gérer les clés SSH" pour trouver la clé dans la section "**Clés privées**" et cliquer "**Afficher/Télécharger**" pour rapatrier le fichier vers le PC.
4. Clé privée téléchargée et rangée dans `C:\Users\DamienRockstar\.ssh\` (dossier standard utilisé par le client SSH intégré à Windows — pas besoin de PuTTY ni de conversion au format **.ppk**).
5. Clé publique **autorisée** (cPanel → Clés publiques → **passerelle_pc_damien** → Gérer → Authorize) — sans cette étape, la clé existe mais ne donne accès à rien.

💡 **Comprendre : générer ≠ autoriser.** Deux actions bien distinctes étaient nécessaires : *générer* crée la paire de clés, *autoriser* dit explicitement au serveur "fais confiance à cette clé publique". Oublier l'autorisation est une erreur courante — la clé semble prête mais la connexion échouera quand même.

Informations de connexion :

- Nom d'hôte : **<HOTE_O2SWITCH>** (adresse IP : **<IP_SERVEUR>**)
- Utilisateur : **<COMPTE>**
- Port : **22** (standard)

Étape supplémentaire découverte le 07/09/2026 — Autorisation SSH par IP : o2switch bloque le port 22 par défaut pour toute adresse non explicitement autorisée (protection anti-intrusion), en plus de

l'authentification par clé. Sans cette autorisation, la connexion échoue en **Connection timed out** (aucune réponse — différent d'un **Connection refused**, qui indiquerait un port fermé mais un serveur joignable).

 **Comprendre : deux couches de protection indépendantes.** La clé SSH prouve *qui* tu es ; la liste blanche d'IP contrôle *d'où* tu as le droit de te connecter. Retirer l'une ne supprime pas l'autre — les deux doivent être en place.

Procédure : cPanel → barre de recherche "Search Tools" → **"Autorisation SSH"** (outil exclusif o2switch, section Outils Exclusifs) → **"Ajouter une autorisation"** → adresse IP publique actuelle (auto-détectée par l'outil) → port **22 - SSH** (les autres ports proposés — 8888, 27017-27019 — sont des ports alternatifs pour contourner un blocage de port 22 *sortant* côté réseau client ; non nécessaire ici) → Confirmer.

 **Si la connexion venait à retimerouter plus tard** : la cause la plus probable est un changement d'adresse IP publique du domicile de Damien (IP dynamique — cas le plus courant en France pour une offre internet résidentielle sans option "IP fixe"). Réflexe : revenir sur "Autorisation SSH" et ajouter la nouvelle IP actuelle.

Commande de connexion validée :

```
ssh -i C:\Users\DamienRockstar\.ssh\passerelle_pc_damien <COMPTE>@<HOTE_O2SWITCH>
```

(Étape validée le 07/09/2026 : première connexion SSH réussie depuis le PC de Damien, après ajout de l'autorisation IP. Prompt serveur obtenu : [**<COMPTE>@<SERVEUR>** ~]\$.)

2. Préparation de l'hébergement (Node.js Selector)

Rôle de l'outil : contrairement à PHP (exécuté à la demande), Node.js doit tourner en **processus persistant**. Node.js Selector (basé sur *Passenger*) gère ce processus sur l'hébergement mutualisé o2switch. Vérifié dans le code (08/09/2026) : le projet est un **monorepo pnpm** avec deux applications indépendantes — **artifacts/api-server/** (backend Express, seule partie gérée par Node.js Selector) et **artifacts/onboardcc/** (frontend React/Vite, buildé en fichiers statiques servis directement par Apache — voir étape 6).

 **Architecture retenue** : le frontend attend l'API sur le **même domaine**, sous-chemin **/api** (**VITE_API_BASE_URL=/api** dans le code). Donc **passerelle.dm-informatique.fr/** doit afficher l'interface React (fichiers statiques), et **passerelle.dm-informatique.fr/api/*** doit être redirigé vers le backend Node.js — les deux cohabitent sur le même sous-domaine.

Configuration de l'application (cPanel → Setup Node.js App → Créer une application) :

Champ	Valeur	Pourquoi
Version de Node.js	24.19.0 (la plus récente proposée)	Aligné avec la version utilisée en développement — évite les écarts de version
Mode d'application	Production	NODE_ENV=production, conditions proches du réel (rôle VAL)
Racine de l'application	passerelle-api	Dossier contenant le monorepo cloné (voir ci-dessous)

Champ	Valeur	Pourquoi
URL de l'application	<code>passerelle.dm-informatique.fr</code> + sous-chemin <code>api</code>	Isole le backend sous <code>/api</code> , laisse <code>/</code> libre pour le frontend
Fichier de démarrage	<code>artifacts/api-server/dist/index.mjs</code>	Point d'entrée compilé du backend

Récupération du code sur le serveur :

```
git clone https://github.com/DamienMoreauConsultant/onboardcc.git ~/passerelle-api
```

Aucune authentification n'est nécessaire : le dépôt `onboardcc` est public, le `git clone` (et les `git pull` suivants) fonctionnent sans identifiant ni jeton.

Le dossier `~/passerelle-api` n'a pas besoin d'être créé au préalable : `git clone` le crée lui-même s'il n'existe pas. Attention au cas inverse, où il existe déjà :

Situation	Ce qui se passe	Que faire
Le dossier n'existe pas	<code>git clone</code> le crée et y copie tout le dépôt	Rien de plus
Le dossier existe mais est vide (par exemple créé par cPanel à la création de l'application, ci-dessus)	<code>git clone</code> fonctionne	Vérifier avant avec <code>ls -A ~/passerelle-api</code> : aucune ligne ne doit s'afficher
Le dossier existe et n'est pas vide	<code>git clone</code> refuse (<code>destination path ... already exists and is not an empty directory</code>) : il ne remplace ni ne fusionne rien	Voir ci-dessous

Si le dossier contient déjà une installation :

- **Il s'agit seulement de mettre à jour le code** : ne pas recloner. Se placer dans le dossier et récupérer les nouveautés (`cd ~/passerelle-api && git status && git pull`) — voir 2-procedure_maintenance_passerelle.md §2.
- **Il s'agit de repartir de zéro** : ne pas supprimer le dossier à la légère — il contient aussi le sous-dossier `uploads/` (les pièces jointes des candidats). Le mettre de côté (`mv ~/passerelle-api ~/passerelle-api_ancien`), cloner, recopier `uploads/` si les pièces jointes doivent être conservées, puis ne supprimer l'ancien dossier qu'une fois la nouvelle installation vérifiée.

💡 **C'est quoi une "dépendance", npm, pnpm ?** Le code de Passerelle n'est pas écrit à 100% par Damien : il s'appuie sur des centaines de petites briques déjà écrites par d'autres (afficher un formulaire, se connecter à PostgreSQL, envoyer un email...). Chacune de ces briques est une **dépendance**, listée dans `package.json`. **npm** ("Node Package Manager") est l'outil standard livré avec Node.js pour télécharger ces briques depuis leur registre en ligne et les ranger dans un dossier `node_modules`. **pnpm** fait exactement le même métier que npm, en plus rapide et en plus économe en

espace disque (il stocke chaque brique une seule fois sur le disque et la relie par raccourcis à tous les projets qui l'utilisent, au lieu de la dupliquer) — c'est pour ça que ce projet l'impose. Concrètement : `pnpm install` lit `package.json` et télécharge tout ce qu'il faut ; c'est la toute première commande à lancer sur un nouvel environnement (serveur, ou PC de dev), avant de pouvoir builder ou démarrer l'application.

⚠ **Ne pas utiliser le bouton "Exécuter NPM Install"** proposé par l'interface cPanel — le `package.json` racine bloque volontairement `npm` ("Use `pnpm` instead"), ce projet impose `pnpm`. Installation manuelle via SSH, dans l'environnement virtuel dédié créé par cPanel :

```
source /home/<COMPTE>/nodeenv/passerelle-api/24/bin/activate && cd
/home/<COMPTE>/passerelle-api
npm install -g pnpm
pnpm install
pnpm approve-builds # approuver esbuild (script postinstall légitime, bloqué par
défaut par pnpm pour la sécurité)
pnpm install        # relancer pour exécuter le script approuvé
cd artifacts/api-server
pnpm run build      # génère dist/index.mjs
```

⚠ **Réflexe obligatoire après `pnpm approve-builds` : vérifier `git status`.** Cette commande peut modifier `pnpm-workspace.yaml` **localement sur ce serveur** (elle y a écrit l'approbation). Si `git status` montre ce fichier modifié, **ne pas laisser cet écart ici** : il faut le reporter à la source (le committer et le pousser depuis un PC de développement, puis le repuller sur tous les environnements), pas le laisser traîner comme une différence locale au serveur.

Pourquoi c'est important : un serveur ne doit jamais avoir de modification non committée (voir §1.1) — sinon le prochain `git pull` de maintenance échoue avec `error: Your local changes ... would be overwritten by merge` (vécu le 15/09/2026 sur o2switch, exactement pour ce fichier : `pnpm approve-builds` avait modifié `pnpm-workspace.yaml` lors de cette installation initiale, sans que le changement soit rapatrié vers GitHub — voir 2-procedure_maintenance_passerelle.md §2.4 pour la procédure de résolution si ça se reproduit). **Ne jamais résoudre ça en donnant au serveur le droit de `push`** — un serveur compromis pourrait alors pousser du code malveillant sur GitHub ; toujours corriger à la source (PC local), en sens unique.

(Étape validée le 08/09/2026 : application créée, dépendances installées (526 packages), `dist/index.mjs` généré avec succès. L'application ne répond pas encore — il manque la base de données (étape 3) et les variables d'environnement (étape 4) avant de pouvoir démarrer réellement.)

3. Base de données PostgreSQL

⚠ **Règle valable pour TOUTE commande `psql` de cette section (§3.1 à §3.5), sur tout serveur — o2switch, un futur serveur DCC, ou en local** : si `psql` est lancé **depuis un poste Windows** (même pour se connecter à un serveur distant), forcer l'encodage client avant toute commande de chargement de fichier (`\i` ou `-f`) :

```
$env:PGCLIENTENCODING = "UTF8"
```

Pourquoi c'est nécessaire : sur Windows, `psql` détecte automatiquement le code page de la console (souvent 850 ou 1252, jamais UTF-8 par défaut) et l'utilise comme encodage client sans rien demander. Si le fichier chargé contient des caractères accentués (noms de pays, libellés de référentiels...), PostgreSQL les interprète alors deux fois dans le mauvais sens et les corrompt silencieusement en base (é devient Ã©) — **aucune erreur n'est levée**, la corruption n'est découverte que plus tard, à l'affichage. Bug rencontré et diagnostiqué le 18/09/2026 en local. Cette variable n'a **aucun effet indésirable** si elle est déjà correcte (Linux/SSH le sont déjà par défaut) — on peut la mettre systématiquement, par réflexe, sans avoir à se demander si elle est nécessaire dans ce cas précis.

💡 Ce risque ne s'est **pas produit** lors du déploiement o2switch du 08/09/2026 (§3.3-3.4 ci-dessous) car `psql` y est lancé **via une session SSH sur le serveur Linux lui-même** (`psql -h localhost -U ...`, après connexion SSH) — un terminal Linux est en UTF-8 par défaut. Le risque réapparaît uniquement si quelqu'un se connecte un jour avec un `psql` installé **localement sur un poste Windows**, pointé directement vers un serveur distant (au lieu de passer par SSH) — scénario à éviter, mais qu'un futur prestataire DCC pourrait choisir sans le savoir. D'où la règle générale ci-dessus, à appliquer par précaution.

3.1 Vérifier la version PostgreSQL cible AVANT de générer quoi que ce soit

⚠ **Leçon apprise le 08/09/2026** : toujours vérifier la version PostgreSQL du serveur cible en premier, avant de générer un dump de schéma — ça évite un aller-retour complet.

```
SELECT version();
```

💡 **Pourquoi c'est critique** : le code est développé et testé avec PostgreSQL 16. Si le serveur cible est plus ancien (o2switch : **PostgreSQL 9.6.22**, sorti en 2016, non maintenu depuis 2021), un export généré tel quel par `pg_dump` moderne contient une syntaxe que l'ancien serveur ne comprend pas (voir §3.3). Ne pas supposer que la version cible est récente — la DCC pourrait elle aussi se retrouver avec un hébergement mutualisé imposant une vieille version : refaire ce contrôle systématiquement sur tout nouvel environnement, ne jamais réutiliser un script à l'aveugle sous prétexte qu'il a marché ailleurs.

3.2 Créer la base, l'utilisateur, et les droits (cPanel → Assistant de base de données PostgreSQL)

Utiliser l'**Assistant** plutôt que l'interface manuelle "Bases de données PostgreSQL" — il enchaîne création de la base, création de l'utilisateur, et attribution des droits, sans risquer d'oublier une étape (contrairement à la version manuelle, où il faut penser aux trois séparément).

Sur un hébergement mutualisé, la base et l'utilisateur sont automatiquement préfixés par l'identifiant du compte (ex. `passerelle` → `<COMPTE>_passerelle`).

(Créé le 08/09/2026 : base `<COMPTE>_passerelle`, utilisateur `<COMPTE>_passerelle_app`.)

⚠ Point découvert : l'assistant donne tous les droits sur la base à l'utilisateur applicatif, mais **pas la propriété du schéma public lui-même** — un `DROP SCHEMA public CASCADE` échoue avec **doit être le propriétaire du schéma public**. Pour repartir d'une base propre sans cette propriété, supprimer individuellement les tables/séquences existantes plutôt que le schéma :

```
DO $$
DECLARE r RECORD;
BEGIN
  FOR r IN (SELECT tablename FROM pg_tables WHERE schemaname = 'public') LOOP
    EXECUTE 'DROP TABLE IF EXISTS public.' || quote_ident(r.tablename) || '
CASCADE';
  END LOOP;
  FOR r IN (SELECT sequence_name FROM information_schema.sequences WHERE
sequence_schema = 'public') LOOP
    EXECUTE 'DROP SEQUENCE IF EXISTS public.' || quote_ident(r.sequence_name) || '
CASCADE';
  END LOOP;
END $$;
```

3.3 Préparer le fichier d'installation de la base (schéma + référentiels)

Un **fichier unique** contient tout ce qu'il faut pour une base neuve : le schéma complet (toutes les migrations déjà appliquées) et les référentiels validés DCC (pays, compétences, langues...), **sans aucune donnée personnelle** (RGPD : jamais de vrais candidats ni de données de test dans ce fichier) :

`documentation/prompt_0_sql_versionAvantProd_20260918.sql` (dans le dépôt de code).

- **PostgreSQL 10 ou plus récent** (§3.1) : le fichier s'utilise tel quel.
- **PostgreSQL < 10** (cas d'o2switch : 9.6.22) : utiliser la version convertie `prompt_0_sql_versionAvantProd_20260918_pg96.sql`, à côté du fichier d'origine. Elle ne diffère que par la suppression de 4 éléments, inconnus des anciens serveurs :
 - `\restrict / \unrestrict` (marqueurs `pg_dump` récents) ;
 - `SET default_table_access_method = heap` ; (paramètre PostgreSQL 12+) ;
 - `AS integer` dans les `CREATE SEQUENCE` (clause PostgreSQL 10+) ;
 - `ALTER ... OWNER TO postgres` ; (le rôle `postgres` n'existe pas sur un compte mutualisé ; sans conséquence, les objets restent la propriété de l'utilisateur applicatif).

Pour régénérer la version convertie après une évolution du fichier d'origine (ne **jamais** l'éditer à la main), depuis le dossier `documentation/` :

```
sed -e '/^\s*\restrict/d;/^\s*\unrestrict/d' -e '/^SET default_table_access_method/d'
-e '/^      AS integer$/d' -e '/OWNER TO postgres;/d' \
  prompt_0_sql_versionAvantProd_20260918.sql >
  prompt_0_sql_versionAvantProd_20260918_pg96.sql
```

Transfert vers le serveur par `scp` (pas besoin de Git pour ce fichier ponctuel) :

```
scp -i C:\Users\DamienRockstar\.ssh\passerelle_pc_damien
"chemin\vers\prompt_0_sql_versionAvantProd_20260918_pg96.sql"
<COMPTE>@<HOTE_O2SWITCH>:~/passerelle-api/backups/
```

3.4 Charger le fichier dans la base

Connecté au serveur en SSH (§1.2), puis à la base :

```
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

```
\i backups/prompt_0_sql_versionAvantProd_20260918_pg96.sql
```

Le fichier est transactionnel : lire le résultat, puis vérifier que la base contient bien les référentiels (au minimum : 6 régions, 42 pays, 54 compétences, 11 états candidat, 36 aides contextuelles). Une fois chargé, il manque encore le premier compte admin (aucune donnée de compte dans ce fichier) — voir §8.1.

⚠ Une erreur affichée en cours de chargement (ex. `syntax error, role "postgres" does not exist`) signifie presque toujours que la version d'origine a été chargée sur un serveur trop ancien : repartir d'une base vide (§3.2) avec la version `_pg96`.

3.5 Cas particulier : poste de développement local (ajouté le 18/09/2026)

Pour une base de dev locale (PostgreSQL 16 — donc pas besoin de la conversion de compatibilité du §3.3), §3.3+3.4 se résument en **un seul fichier tout-en-un**, dans le dépôt de code (pas ce dépôt-ci) :

`Passerelle_code/onboardcc/documentation/prompt_0_sql_versionAvantProd_20260918.sql` — combine le dernier schéma à jour (dump du 08/09/2026, toutes migrations incluses) et les référentiels validés DCC. Aucune connexion à un serveur distant nécessaire.

💡 **Déplacé le 18/09/2026** de `Documents/3 - BDD/` (ce dépôt-ci) vers `Passerelle_code/onboardcc/documentation/` (le dépôt de code) — pendant le MVP initial, un seul fichier fait foi, maintenu dans le dépôt qui est effectivement poussé sur GitHub. À rouvrir et actualiser à chaque changement de schéma ou de référentiel constaté en test, plutôt que de laisser plusieurs copies diverger.

⚠ **Poste Windows : appliquer la règle d'encodage donnée en tête de la section 3** avant le `-f` ci-dessous (`$env:PGCLIENTENCODING = "UTF8"`) — c'est justement le cas concret où ce bug a été rencontré le 18/09/2026.

```
& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -c "CREATE DATABASE
nom_de_la_base;"
$env:PGCLIENTENCODING = "UTF8"
& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d nom_de_la_base -f
```

```
"C:\Users\DamienRockstar\Documents\Passerelle_code\onboardcc\documentation\prompt_0_sql_versionAvantProd_20260918.sql"
```

⚠ **Quand le schéma évolue** (nouvelle migration dans `artifacts/api-server/migrations/`), ce fichier est mis à jour en même temps, avec une ligne de suivi en tête de fichier : ne **jamais** rejouer les migrations une par une sur une base neuve. Si la version `_pg96` existe, la régénérer (§3.3).

Une fois chargé, il manque encore le premier compte admin (aucune donnée de compte dans ce fichier) — voir §8.1 ci-dessous.

4. Variables d'environnement de production

La **liste de référence** des variables est le fichier `artifacts/api-server/.env.example` du dépôt de code : il est tenu à jour avec le code (chaque variable `MATCH_SCORE_*` lue par `matching.ts` y figure). Une seule variable manquante bloque tous les calculs de score.

Le serveur charge un fichier `.env` s'il en trouve un (pratique en développement), sans jamais écraser une variable déjà définie. **Sur o2switch, on continue à passer par le panneau cPanel → Setup Node.js App → Environment variables**, pas par un fichier `.env` sur le serveur.

Variables à renseigner : toutes celles de `.env.example` — le barème `MATCH_SCORE_*` (à copier tel quel) et, propres à l'environnement : `DATABASE_URL`, `JWT_SECRET` (généré via `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`, propre à cet environnement — ne jamais réutiliser celui d'un autre), `JWT_EXPIRE_HOURS=8`, `FRONTEND_URL=https://passerelle.dm-informatique.fr`, `UPLOAD_DIR=/home/<COMPTE>/passerelle-api/uploads`, `PIECE_JOINTE_STORAGE_URL=https://passerelle.dm-informatique.fr/uploads/`, et `PORT` (voir §7).

Les `SMTP_*` sont volontairement absents — reportés à l'étape 6.

💡 **Pour une future installation complète depuis zéro (nouveau serveur DCC)** : partir de `artifacts/api-server/.env.example` (source à jour) — ne pas reprendre une ancienne liste : copier chaque ligne dans cPanel → Environment variables, en adaptant uniquement celles propres à l'environnement (`DATABASE_URL`, `JWT_SECRET`, `FRONTEND_URL`, `UPLOAD_DIR`, `PIECE_JOINTE_STORAGE_URL`, `SMTP_*`) ; toutes les `MATCH_SCORE_*` se copient telles quelles (valeurs métier identiques quel que soit l'environnement).

(Étape validée le 08/09/2026 pour tout sauf SMTP.)

5. Stockage des pièces jointes (dossier `uploads/`)

Rappel du principe acté le 06/09/2026 (issue P4_35) : l'écriture du fichier est toujours locale au serveur qui exécute l'application, seule l'URL publique (`PIECE_JOINTE_STORAGE_URL`) change entre environnements. Aucune modification de code nécessaire.

```
mkdir -p /home/<COMPTE>/passerelle-api/uploads
chmod 755 /home/<COMPTE>/passerelle-api/uploads
echo "Options -Indexes" > /home/<COMPTE>/passerelle-api/uploads/.htaccess
```

💡 **Options -Indexes** : empêche de lister le contenu du dossier en y naviguant directement dans un navigateur — chaque pièce jointe reste accessible par son lien précis, jamais en vrac.

(Étape validée le 08/09/2026 : dossier créé avec les permissions **755** et le **.htaccess**, chemin identique à **UPLOAD_DIR** déjà défini à l'étape 4.)

6. Configuration du serveur SMTP (envoi d'emails)

⚠ **Distinction importante (09/09/2026)** : cette étape configure l'**infrastructure** d'envoi d'email (un vrai compte SMTP fonctionnel) — nécessaire dès maintenant car le mécanisme de **réinitialisation de mot de passe est déjà codé** et bloque le test de bout en bout (§8) sans elle. Les **notifications applicatives** candidat/CM (invitation à interagir, accepter/décliner une opportunité, notification CM) listées ci-dessous ne sont, elles, **pas encore développées** — configurer le SMTP ne les fera pas apparaître comme par magie, ce sera un chantier de code séparé, plus tard.

Périmètre identifié (variables déjà prévues dans **.env.example** du backend : **SMTP_HOST**, **SMTP_PORT**, **SMTP_USER**, **SMTP_PASSWORD**, **SMTP_FROM** — actuellement en **pending_smtp_configuration**, jamais configuré en développement) :

- **Réinitialisation de mot de passe** — lien d'expiration envoyé par email — ☒ déjà codé côté backend (**POST /api/auth/mot-de-passe-oublie**, voir **.agents/memory/password-reset-safety.md**).
- **Emails candidat** (*pas encore développé*) — invitation à interagir : saisir sa fiche de vœux, consulter et accepter/décliner une opportunité mise en lien.
- **Emails Chargé de Mission** (*pas encore développé*) — notification de réception d'un candidat sur un poste, à approuver ou rejeter.

Mise en place (09/09/2026)

1. Créer un compte email dans cPanel → "**Comptes de messagerie**" — ex. **noreply@passerelle.dm-informatique.fr**, mot de passe respectant la règle de caractères en tête de ce document.
2. Récupérer les réglages serveur via "**Configurer le client de messagerie**" (lien affiché en face du compte créé) : nom d'hôte SMTP, port, type de chiffrement.
3. Renseigner les 5 variables d'environnement dans cPanel (même panneau que les autres, voir §4) : **SMTP_HOST**, **SMTP_PORT**, **SMTP_USER** (l'adresse email complète), **SMTP_PASSWORD**, **SMTP_FROM**.

À compléter une fois cette étape terminée et testée.

7. Déploiement du code et démarrage de l'application

⚠ **Rappel : avant tout **git clone** ou **git pull** sur le serveur, vérifier que le dépôt GitHub est bien à jour** de la version à installer (voir §1.1).

⚠ **Découverte critique du 08/09/2026** : contrairement à l'usage habituel de Passenger (qui fournit normalement lui-même la variable **PORT** à l'application), **ce n'est pas automatique sur cet hébergement o2switch**. Sans elle, l'application plante immédiatement au démarrage (**Error: PORT environment variable is required but was not provided.**), ce qui se traduit côté cPanel par le message générique et peu informatif "**Could not spawn process... The application process exited prematurely.**" (visible dans cPanel → Erreurs), sans aucun détail exploitable.

Méthode de diagnostic qui a fonctionné : reproduire le lancement manuellement en SSH, avec les mêmes variables et le même dossier que Passenger (**Racine de l'application**), pour voir le vrai message d'erreur Node.js — invisible autrement sur cet hébergement mutualisé (pas d'accès aux logs Passenger détaillés) :

```
cd /home/<COMPTE>/passerelle-api
export PORT=3000
export DATABASE_URL='...' # guillemets SIMPLES obligatoires (mots de passe avec ! ou $)
export JWT_SECRET='...'
# ... autres variables ...
node artifacts/api-server/dist/index.mjs
```

Correctif : ajouter manuellement **PORT=3000** (ou toute valeur libre) dans cPanel → Environment variables. Aucun conflit constaté avec le proxy interne de Passenger — a fonctionné immédiatement, sans même nécessiter de redémarrage manuel (l'enregistrement des variables semble redémarrer l'application automatiquement sur ce cPanel).

(Étape validée le 08/09/2026 pour le backend : <https://passerelle.dm-informatique.fr/api/healthz> répond `{"status": "ok"}`. **Reste à faire** : build et déploiement du frontend React/Vite — voir ci-dessous, le domaine racine ne sert pour l'instant que l'API.)

7.1 Frontend

L'application est montée à la racine du domaine et gère elle-même **/api/*** et **/uploads/*** (voir §2) — tout le reste (/ , les routes de l'interface React, les fichiers statiques) est servi par **Apache directement**, pas par Node.

1. Build (depuis le dossier du paquet directement, pas via **pnpm --filter** — voir §2 pour la raison du lien symbolique **node_modules**) :

```
cd /home/<COMPTE>/passerelle-api/artifacts/onboardcc
export PORT=5000
export BASE_PATH=/
export VITE_API_BASE_URL=/api
pnpm run build
```

💡 **BASE_PATH=** est important : il détermine comment les chemins vers les fichiers JS/CSS sont écrits dans la page générée. Une mauvaise valeur casserait le chargement de l'interface en production. **PORT** est obligatoire pour que le fichier de config s'évalue, sans impact réel sur le résultat.

Fichiers générés dans **dist/public/** (pas **dist/** — voir **vite.config.ts**).

2. Dépôt des fichiers dans le dossier public du domaine :

```
cp -r /home/<COMPTE>/passerelle-api/artifacts/onboardcc/dist/public/*
/home/<COMPTE>/passerelle.dm-informatique.fr/
```

3. Règles Apache — ajoutées (jamais en remplaçant) au `.htaccess` déjà généré par Node.js Selector dans ce même dossier :

```
cat >> /home/<COMPTE>/passerelle.dm-informatique.fr/.htaccess << 'EOF'

# --- Passerelle : cohabitation frontend statique + backend Node (09/09/2026) ---
RewriteEngine On
RewriteCond %{REQUEST_URI} ^/(api|uploads)/ [OR]
RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^ - [L]
RewriteRule ^ /index.html [L]
EOF
```

💡 Passenger sert un vrai fichier statique existant directement (sans passer par Node) — ce bloc ajoute un filet de sécurité explicite : `/api/uploads` ou un fichier/dossier réel → inchangé ; sinon → repli sur `index.html` (indispensable pour les routes internes React, ex. `/login`).

(Étape validée le 09/09/2026 : <https://passerelle.dm-informatique.fr/> affiche l'interface Passerelle.)

7.2 Sécurisation du serveur (préconisations à l'installation)

Ajouté le 17/09/2026, à la suite d'un audit de sécurité du support de maintenance (voir [conception.md](#) §7.4.7 pour la synthèse complète face à l'OWASP Top 10). Checklist à parcourir **avant toute mise à disposition réelle**, sur ce serveur comme sur tout futur serveur DCC — une partie est déjà en place sur o2switch (référéncée ci-dessous vers l'étape correspondante), le reste reste à vérifier explicitement sur chaque nouvel environnement.

Accès réseau

- ☐ Seuls les ports strictement nécessaires sont ouverts (80/443 web, 22 SSH) — aucun autre port du serveur applicatif ou de la base ne doit être exposé publiquement.
- ☒ SSH : authentification par clé uniquement (pas par mot de passe) + liste blanche d'IP côté hébergeur — voir §1.2, déjà en place sur o2switch.
- ☐ PostgreSQL n'écoute que sur `localhost`, jamais accessible depuis l'extérieur — cohérent avec la contrainte retenue dans [conception.md](#) §7.2 ("Accès local uniquement sur mutualisé"), à reconfirmer explicitement sur tout nouvel hébergement.

HTTPS / DNS

- ☐ Certificat SSL valide et actif (Let's Encrypt via AutoSSL ou équivalent) — voir §9, point déjà identifié comme nécessitant une vérification manuelle sur tout nouveau domaine.
- ☐ Redirection automatique HTTP → HTTPS activée côté serveur.
- ☐ HSTS actif en production : déjà posé au niveau applicatif (`app.ts`, voir zoom Sécurité du support de maintenance) — vérifier qu'aucune configuration serveur ne l'écrase ou ne l'annule.
- ☐ DNS : uniquement les enregistrements strictement nécessaires (A/AAAA vers le bon serveur, MX si email) — supprimer tout enregistrement obsolète pointant vers un ancien hébergement (risque de détournement, dit "subdomain takeover").
- ☐ Si l'hébergeur DNS le permet, activer DNSSEC (protection contre la falsification de réponses DNS).

⚠ Non couvert à ce jour (17/09/2026) : ce point "HTTPS / DNS" a été identifié comme absent du support de maintenance lors d'un audit de sécurité (*conception.md* §7.4.7). L'hébergement définitif DCC n'étant pas encore choisi (voir §9 et la diapositive "Les environnements retenus"), cette checklist sera à exécuter par la DCC ou son prestataire au moment du choix de cet hébergement.

Fichiers et permissions

- ☐ Le dossier `.git` du dépôt cloné sur le serveur n'est jamais accessible depuis le web — vérifier qu'il n'est pas dans le dossier public, ou le bloquer explicitement via `.htaccess` sinon.
- ☒ Aucune donnée sensible (`.env`, clés, mots de passe) dans un dossier servi publiquement : les secrets de ce projet passent uniquement par les variables d'environnement cPanel, jamais par un fichier `.env` sur le serveur — voir §4.
- ☒ `uploads/ : Options -Indexes` actif pour empêcher le listing — voir §5. Reprendre ce réflexe sur tout nouveau dossier de stockage de fichiers.
- ☐ Permissions fichiers/dossiers au strict nécessaire (jamais de `chmod 777`).

Comptes et secrets

- ☐ Aucun compte de test/démo actif avec un mot de passe faible ou connu une fois en production réelle — le script de nettoyage des comptes `TEST_*` (voir §8.2) doit être exécuté avant toute mise à disposition réelle.
- ☒ `JWT_SECRET` propre à chaque environnement, jamais réutilisé d'un environnement à l'autre — voir §4.
- ☒ Règle de mots de passe du projet respectée (voir encadré en tête de ce document).

Mises à jour et composants

- ☐ Version de PostgreSQL et de Node.js du serveur cible vérifiée avant installation (voir §3.1), et signalée explicitement si obsolète/non maintenue. (*Cas connu : o2switch impose PostgreSQL 9.6.22, non maintenu depuis 2021 — accepté comme contrainte de cet hébergement mutualisé précis ; à ne pas reproduire sans réflexion sur un futur serveur DCC si un choix de version est possible.*)
- ☒ `pnpm-lock.yaml` versionné et jamais modifié manuellement sur un serveur — garantit que les mêmes versions de dépendances sont installées sur tous les environnements — voir l'avertissement §2 sur `pnpm approve-builds`.

Limitation des tentatives de connexion

- ☐ Mettre en place une protection anti-brute-force sur `/api/auth/login` (ex. limitation du nombre de tentatives par IP/compte). (*Reste à faire côté code, pas encore implémenté au 17/09/2026 — identifié lors de l'audit de sécurité, conception.md §7.4.7.*)

8. Test de bout en bout en production

Sur données `TEST_*`, avec restauration après coup, conformément à la discipline de test suivie tout au long du projet.

8.1 Création du premier compte admin de l'application

⚠ Brouillon non encore validé (10/09/2026) — cette étape n'a jamais été exécutée depuis une base vide (sur o2switch, le premier compte admin existait déjà avant que cette procédure ne soit rédigée, sa

création précise n'a pas été tracée). Damien va tester cette méthode sur son environnement de dev local et la corriger ici une fois validée.

Aucune interface ne permet de créer le tout premier compte admin (il faut déjà être connecté en admin pour créer des utilisateurs) — il faut donc l'insérer directement en base, avec un mot de passe **haché en bcrypt** (jamais en clair, même pour un compte de test).

1. Générer le hash bcrypt du mot de passe choisi, depuis `artifacts/api-server` (où `bcryptjs` est déjà installé) :

```
cd artifacts\api-server
node -e "const bcrypt=require('bcryptjs'); bcrypt.hash('MOT_DE_PASSE_ADMIN', 12).then(h=>console.log(h))"
```

💡 Respecter la règle de caractères en tête de ce document. `12` = nombre d'itérations bcrypt, cohérent avec `hashTestPasswords.ts` (voir ce script pour référence).

2. Créer le contact, puis l'utilisateur, connecté à la base (locale ou o2switch) via `psql` :

```
INSERT INTO contact (crm_key, role, genre, nom_contact, prenom_contact,
tel_contact, email_contact)
VALUES ('TEST_ADMIN_01', 'ADMIN', 'F', 'NOM_A_CHOISIR', 'PRENOM_A_CHOISIR',
'0600000000', 'admin.test@passerelle.local')
RETURNING id_contact;
```

Noter l'`id_contact` renvoyé, puis :

```
INSERT INTO user_ (login, password, id_contact, role_applicatif)
VALUES ('admin.test', 'HASH_GENERE_ETAPE_1', ID_CONTACT_NOTE_CI_DESSUS, 'ADMIN');
```

💡 `role_applicatif` doit être exactement `'ADMIN'` (contrainte `chk_role_applicatif`, migration `014_role_applicatif.sql`) — c'est cette colonne, pas `contact.role`, qui conditionne les droits réels dans l'application.

3. Vérifier : connexion sur l'écran de login avec `admin.test` / le mot de passe choisi à l'étape 1 → accès à `/admin/referentiels`.

8.2 Création des comptes entreprises de test (avec le compte admin)

Une fois connecté avec le compte admin (§8.1), créer les comptes Recruteur/CM de test via l'écran **Admin** → **"Nouvel utilisateur"** → **"Créer et inviter"** — voir sanity check #2 en Annexe A.2 pour le détail de cette manipulation à l'écran.

⚠️ **À prévoir avant la prochaine recette complète** (pas encore fait au 10/09/2026) :

- Un **jeu de fichiers CSV dédié** à cette validation (postes + candidats), distinct des fichiers de démo dans **3 - Test data/** — pour ne pas mélanger données de démonstration commerciale et données de validation technique.
- Un **script de nettoyage** à exécuter en toute fin de recette, qui supprime automatiquement toutes les **opportunités** générées à partir de ces candidats/postes de test, puis les candidats et postes eux-mêmes — en s'appuyant sur le préfixe **TEST_** des **crm_key**, comme pour toute donnée de test de ce projet (cohérent avec la discipline **TEST_*** déjà en place). Ce script n'existe pas encore — à écrire avant la prochaine recette complète.

8.3 Liste des sanity checks à valider avant mise à disposition

Liste des fonctionnalités à valider avant de donner accès à l'application (pas le détail d'exécution ni les statuts — voir Annexe A.2 pour la dernière recette exécutée avec son résultat) :

1. Connexion administrateur
2. Création d'un compte Recruteur + invitation
3. Réception réelle de l'email d'invitation
4. Connexion avec le nouveau compte Recruteur
5. Import de postes (CSV test)
6. Import de candidats (CSV test)
7. Candidat : édition de la fiche de vœux (listes déroulantes + bouton "Éditer vœux")
8. Génération automatique des opportunités (R1/R2)
9. Parcours recruteur/CM jusqu'à affectation (mise en lien → accord de principe → accord définitif → affecté)
10. Exécution du script de nettoyage (§8.2) et vérification que la base est revenue à un état propre

9. Point de vigilance pour une future migration DCC

À compléter au fur et à mesure — ce que la DCC devra spécifiquement adapter par rapport à la procédure o2switch (nom de domaine, éventuel changement d'hébergeur, SMTP propre à leur organisation, etc.).

- **Variable PORT (08/09/2026)** : sur o2switch, Passenger ne la fournit pas automatiquement à l'application — il a fallu l'ajouter manuellement (§7). Si le futur serveur DCC utilise aussi Passenger/cPanel (même famille d'hébergement mutualisé), tester ce point en premier en cas d'échec de démarrage : reproduire le lancement en SSH (voir méthode §7) pour voir le vrai message d'erreur, largement plus parlant que l'écran générique Passenger.
- **Version PostgreSQL (08/09/2026)** : o2switch est resté en 9.6 (2016, non maintenu) — toujours vérifier **SELECT version();** avant de générer un export de schéma (voir §3.1).
- **Montage de l'application à la racine du domaine, pas sous /api (08/09/2026)** : le backend gère lui-même les préfixes **/api** et **/uploads** en interne — voir §2 et §7.1 pour le raisonnement complet et la configuration Apache nécessaire pour cohabiter avec le frontend.
- **Certificat SSL auto-signé sur un nouveau domaine (09/09/2026)** : AutoSSL n'émet jamais de certificat Let's Encrypt automatiquement s'il en trouve déjà un (même auto-signé) — correction manuelle nécessaire à chaque nouveau domaine, voir FAQ "Vérifier le certificat SSL" ci-dessous.
- **Caractères spéciaux dans les mots de passe (09/09/2026)** : voir la règle en tête de ce document — se limiter à lettres/chiffres/_-., pour tout mot de passe créé pour ce projet, quel que soit l'hébergeur cible. Un mot de passe avec **#/?/&!** a cassé successivement le terminal (bash) puis la lecture de **DATABASE_URL**, avec un message d'erreur (**Invalid URL**) très éloigné de la vraie cause.

✓ **Installation terminée.** Pour tout geste de maintenance courante à partir de maintenant (déployer un correctif, sauvegarder, diagnostiquer un signalement utilisateur, sécurité...), voir [2-procedure_maintenance_passerelle.md](#).

Journal des mises à jour de ce document

Date	Modification
07/09/2026	Création du document — structure initiale, sections à compléter au fil de la session.
07/09/2026	Étape 1 terminée : code récupéré (GitHub à jour, clone local dans <code>Passerelle_code\onboardcc\</code>) et accès SSH au serveur o2switch configuré et validé (clé + autorisation IP). Pause de session — reprise prévue à l'étape 2 (Node.js Selector).
08/09/2026	Étapes 2 et 3 terminées : application Node.js créée et déployée (dépendances installées, backend compilé), base PostgreSQL créée avec schéma complet (42 tables) et données de référence (15 tables, référentiel validé DCC) chargées avec succès sur o2switch. Prochaine étape : 4 (variables d'environnement) puis 5 (stockage des pièces jointes) avant de pouvoir démarrer réellement l'application.
08/09/2026	Étapes 4, 5 et 7 (backend) terminées : 45 variables d'environnement saisies, dossier <code>uploads/</code> créé, application Node.js démarrée avec succès après diagnostic et correctif de la variable <code>PORT</code> manquante — https://passerelle.dm-informatique.fr/api/healthz répond <code>{"status":"ok"}</code> . Pause de session. Reprise prévue : finir le déploiement (frontend — §7.1, puis étape 8 test de bout en bout), avant d'enchaîner sur la préparation de la soutenance et l'installation de l'environnement de développement local (PC de Damien).
15/09/2026	Document scindé en 2 (demande de Damien) : ce document ne garde que l'installation initiale/réinstallation complète (§1 à §9). L'ancienne "Annexe A — Maintenance courante (FAQ)" est retirée d'ici et redistribuée dans le nouveau 2-procedure_maintenance_passerelle.md , avec en plus la procédure de déploiement d'un correctif (git pull + rebuild + sanity checks), absente jusqu'ici.
17/09/2026	Ajout de la §7.2 "Sécurisation du serveur (préconisations à l'installation)" — checklist réseau/HTTPS-DNS/fichiers/comptes/mises à jour, suite à un audit de sécurité du support de soutenance (voir conception.md §7.4.7). Le volet HTTPS/DNS et la protection anti-brute-force restent à faire, à la charge de la DCC ou de son prestataire pour le premier, côté code pour le second.
25/09/2026	Retrait des éléments propres au poste de développement (procédure destinée à l'exploitant du serveur) : §1.1 réduit à un rappel de vérifier que GitHub est à jour avant <code>pull/clone</code> sur le serveur ; ancien §1.2 (clonage sur le PC de développement) et ancien §3.3 (schéma généré depuis l'environnement de prototype) déplacés dans procedure_local_environnement_dev.md §3 ; §3.3-3.4 réécrits autour du fichier unique <code>prompt_0</code> (avec sa version convertie <code>_pg96</code> pour PostgreSQL < 10) ; §4 : <code>.env.example</code> est la seule référence des variables. §1.3 devient §1.2.
25/09/2026	§2 (récupération du code) : ajout des cas où <code>~/passerelle-api</code> n'existe pas / est vide / n'est pas vide (le <code>git clone</code> ne remplace rien et refuse un dossier non vide ; conduite à

Date**Modification**

tenir pour une mise à jour ou une réinstallation, en préservant `uploads/`) ; suppression des consignes de jeton d'accès, devenues inutiles depuis que le dépôt est public.